

De: Daniel Jorge Alves RA: 0051352421024 - Ciência de dados 2º ciclo

O objetivo deste trabalho foi colocar em prática o que foi aprendido durante uma aula de Álgebra linear nos qual foi apresentado o algoritmo da similaridade de cossenos que é um algoritmo utilizado para medir a similaridade entre textos, ele se define pela seguinte formula:

$$\text{similaridade de cossenos} = (A,B) = (A \cdot B) / (||A|| \cdot ||B||)$$

Onde:

$A \cdot B$ é o produto escalar

$(||A|| \cdot ||B||)$ é o produto das normas

Quanto mais próximo o resultado for de 1 mais similar os textos são
e quanto mais próximo o resultado for de 0 menos similar os textos são

Ele funciona da seguinte forma no código:

Se os vetores representam documentos em um espaço de palavras (como no modelo TF-IDF), a similaridade de cossenos indica quanto esses documentos são semelhantes em termos de conteúdo, independentemente do tamanho do texto.

O que foi utilizado na aplicação do algoritmo:

Top_Anime_data (1).CSV: que é um dataset que tem um overview do top 1000 animes de 2024 esse dataset foi obtido no kaggle e todas as informações coletadas por meio de scrapping do site <https://myanimelist.net>, esse dataset tem vários parâmetros como:

Score(nota do anime com base nos usuarios)

Rank(Ranking do anime com base em sua nota numero de pessoas que já viram o anime)

Japanese Title(Título do anime em japonês)

English Title(Título do anime em inglês)

Status(Se o anime já está completo, em andamento, cancelado, ou pausado)

entre outros parâmetros disponíveis para uso. Porém foram utilizadas apenas os seguintes parâmetros:

English Title - Títulos em inglês dos animes, usados como identificadores para as recomendações.

Description - Sinopses dos animes, usadas para calcular a similaridade dos cossenos.

Score - Notas dos animes, normalizadas para influenciar as recomendações (As notas originais geralmente vão de 1 a 10, enquanto as similaridades dos cossenos variam de

0 a 1 então elas foram “normalizadas” para se adequarem ao trabalho que estava sendo exercido).

Para a programação do código foi utilizada a linguagem Python no programa Visual Studio Code

Para o auxílio na parte da programação foi utilizado o CHATGPT

E Por último também foi utilizado uma lista também vinda do kaggle com **STOPWORDS** em inglês para assim aprimorar a precisão do algoritmo

Explicando o código:

1. Importando Bibliotecas

```
import pandas as pd
import re
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.preprocessing import MinMaxScaler
```

Carrega as bibliotecas necessárias para:

pandas: manipulação de dataframes.

re: expressões regulares para limpar textos.

numpy: operações matemáticas (opcional aqui).

scikit-learn: para vetorizar textos (TF-IDF), calcular similaridades e normalizar dados.

2. Carregando os Dados

```
8 anime_df = pd.read_csv('Top_Anime_data (1).csv')
9
10 with open('stopwords.txt', 'r') as file:
11     stopwords = set(word.strip() for word in file.readlines())
```

Lê o arquivo "Top_Anime_data (1).csv" para um dataframe do pandas, que contém os dados dos animes, incluindo títulos, descrições e notas.

Carrega a lista de stopwords de um arquivo "stopwords.txt" para remover palavras comuns que não agregam significado ao texto (como "and", "the", "of").

3. Pré-processando as Descrições dos Animes

```
13 def preprocess_text(text):
14     text = re.sub(r'^a-zA-Z\s', '', str(text).lower())
15     text = ' '.join(word for word in text.split() if word not in stopwords)
16     return text
17
18 anime_df['processed_description'] = anime_df['Description'].apply(preprocess_text)
```

Remove caracteres especiais e converte todas as letras para minúsculas.

Remove stopwords para focar em palavras mais significativas.
Cria uma nova coluna "processed_description" com o texto limpo.
Converte o texto para minúsculas para uniformizar.

4. Criando a Matriz TF-IDF

```
tfidf = TfidfVectorizer()  
tfidf_matrix = tfidf.fit_transform(anime_df['processed_description'])
```

Transforma os textos das descrições em vetores numéricos com o TF-IDF, que mede a importância de cada palavra em cada descrição.

5. Calculando a Similaridade dos Cossenos

```
cosine_sim_matrix = cosine_similarity(tfidf_matrix)
```

Calcula a similaridade entre todos os pares de animes usando a distância do cosseno.
Resultado: uma matriz onde `matriz[X][Y]` mostra o quão similar o anime X é do anime Y.

6. Normalização das Notas

```
27 scaler = MinMaxScaler()  
28 anime_df['normalized_score'] = scaler.fit_transform(anime_df[['Score']].fillna(0))
```

As notas dos animes são normalizadas para ficarem entre 0 e 1, para que possam ser combinadas com a similaridade

7. Função de Recomendação

```
31 def recommend_animes_with_score(title, anime_df, similarity_matrix, score_weight=0.5, top_n=5):
```

Busca parcial do anime com base no nome fornecido.

Calcula a similaridade com os demais animes.

Retorna os animes mais parecidos com base nas descrições e nas notas.

Busca pelo título (parcial, ignorando maiúsculas e acentos):

```
32     try:  
33  
34         normalized_title = re.sub(r'[^a-zA-Z\s]', '', title.lower())  
35         possible_matches = anime_df[anime_df['English'].fillna('').apply(lambda x: re.sub(r'[^a-zA-Z\s]', '', x.lower()))].str.contains(normalized_title)]
```

Se encontrar, pega o índice e título exato:

```
40         idx = possible_matches.index[0]  
41         title = anime_df.loc[idx, 'English']
```

Calcula a similaridade com todos os outros animes e

Cria uma lista:

```
47         recommendations = []  
48         sim_scores = list(enumerate(similarity_matrix[idx]))  
49         for i, sim in sim_scores:  
50             if i != idx:
```

Gera lista de recomendações:

```
51         recommendations.append((anime_df.iloc[i]['English'], float(sim), anime_df.iloc[i]['Score']))
```

Ordena pelos mais similares:

```
recommendations = sorted(recommendations, key=lambda x: x[1], reverse=True)[:top_n]
```

Imprime o resultado formatado:

```
55     print(f"\nRecomendações para: {title}")
56     print("{:<50} {:<12} {:<6}".format("Anime", "Similarity", "Score"))
57     print("-" * 70)
58     for anime, similarity, score in recommendations:
59         print(f"{anime:<50} {similarity:<12.4f} {score:<6.2f}")
60
61     return recommendations
```

8. Execução com input do usuário

```
63     anime_name = input("Digite o nome do anime para recomendação: ")
64     recommend_animes_with_score(anime_name, anime_df, cosine_sim_matrix, score_weight=0.7, top_n=20)
```

Exemplo de uso:

Neste caso eu queria fazer a pesquisa pelo anime God's Blessing on This Wonderful World! 3 então na pesquisa foi escrito "Kono" para o código buscar pelo anime no dataset

```
Digite o nome do anime para recomendação: Kono
```

Mas o usuário poderia colocar o nome inteiro se quisesse, a pesquisa foi feita dessa maneira para mostrar que o usuário não precisa escrever perfeitamente o nome que ele quer buscar como se ele estivesse tirando esse nome do dataset

```
Recomendações para: KonoSuba: God's Blessing on This Wonderful World! 3
Anime                                     Similarity  Score
-----
KonoSuba: God's Blessing on This Wonderful World! 2 0.2329      8.26
Swallowed Star 3rd Season                      0.1996      8.32
Is It Wrong to Try to Pick Up Girls in a Dungeon? IV Part 2 0.1861      8.24
All Saints Street 3                             0.1634      7.81
That Time I Got Reincarnated as a Slime Season 3  0.1311      8.27
Classroom of the Elite III                     0.1290      8.00
In This Corner of the World                   0.1289      8.23
Kingdom: Season 5                             0.1225      8.75
Black Butler: Public School Arc                0.1135      8.11
Date A Live V                                 0.1055      8.05
nan                                             0.0953      7.89
Swallowed Star 2nd Season                      0.0904      7.99
Swallowed Star 4th Season                     0.0870      8.22
nan                                             0.0869      8.12
The Demon Girl Next Door 2                   0.0838      7.88
nan                                             0.0808      8.02
Fights Break Sphere 5th Season                0.0806      8.11
```

O código retorna os animes com a descrição mais semelhante sendo aqueles que tem a "Similarity" mais próxima de 1 os mais semelhantes

Conclusão:

O trabalho demonstrou com sucesso a aplicação do algoritmo de similaridade de cossenos para recomendar animes com base na semelhança de descrições, utilizando um dataset de preferência pessoal. Essa abordagem permite encontrar animes que possuem descrições textuais mais próximas ao título pesquisado, com a métrica de similaridade variando entre 0 e 1, onde 1 indica máxima semelhança. O uso desse método é particularmente eficaz para recomendar conteúdos semelhantes em grandes conjuntos de dados, como listas de animes, filmes ou artigos, tornando-se uma ferramenta poderosa para sistemas de recomendação.

Referencias:

<https://www.kaggle.com/datasets/rowhitsuami/stopwords>

<https://www.kaggle.com/datasets/bhavyadhingra00020/top-anime-dataset-2024/data>